Information and Software Technology 194 (2026) 108064

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

SRSPSQL: A dual-stage Text-to-SQL framework with semantic rewriting and schema pruning

Jianjun Lei [a], Zhichao Li [a], Ying Wang [b,*]

[a] School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing, 400065, China
[b] School of Artificial Intelligence, Chongqing University of Posts and Telecommunications, Chongqing, 400065, China

ARTICLE INFO

Keywords:
Large language models
Text-to-SQL
Dataset schema
Semantic reconstruction
Dynamic schema pruning

ABSTRACT

Large language models (LLMs) have advanced the Text-to-SQL task significantly. However, existing methods still face challenges in handling natural language ambiguity, limited dataset content, and redundant database schema information. In this paper, we propose SRSPSQL, a dual-stage Text-to-SQL framework that integrates question semantic reconstruction and dynamic schema pruning to eliminate natural language ambiguity and minimize schema search space. Specifically, we design a novel semantic reconstruction mechanism leveraging LLMs' summarization capabilities and concrete database content to eliminate ambiguities and align questions with database structures. Moreover, we develop a full-schema prompt strategy that adopts masked similarity matching for context-aware example selection and integrates database content information to minimize value errors. Furthermore, we propose a concise schema generation method that combines structured SQL information with dynamic schema pruning to focus on key schema and examples, thereby significantly improving the accuracy and efficiency of SQL generation. Experimental results demonstrate that SRSPSQL consistently outperforms baselines, achieving 86.4% and 62.13% execution accuracies (EX) on Spider and Bird, with robust gains on Spider-Syn and Spider-Realistic. Ablation studies further validate the individual and synergistic contributions of each component.

1. Introduction

Text-to-structured query language (Text-to-SQL) is a fundamental technique that enables natural language interaction with databases by automatically translating user queries into structured SQL statements [1]. This allows users to retrieve database information without requiring SQL expertise, thereby greatly simplifying the data retrieval process [2]. In recent research, scholars have systematically reviewed the development of Text-to-SQL techniques, with particular emphasis on the paradigm shift ushered in by the era of large language models (LLMs) [3–5]. However, traditional Text-to-SQL methods still struggle to accurately capture the intent of natural language questions. They also face challenges in precisely aligning referenced entities with the correct tables and columns in the database schema [6]. To overcome these challenges, recent advances in LLMs, which possess powerful semantic understanding and reasoning capabilities, have been increasingly applied to Text-to-SQL tasks [7,8] for enhancing the accuracy and robustness of SQL generation.

Currently, LLM-based Text-to-SQL methods primarily focus on designing in-context learning (ICL) prompts [9–13]. These methods construct prompts that incorporate few-shot examples and database schema information, leveraging the summarization and generalization capabilities of LLMs to effectively perform the Text-to-SQL task. Nevertheless, these prompts typically include substantial irrelevant schema information, which disperses model attention and degrades SQL generation accuracy. Moreover, widely adopted benchmarks such as Spider [14] primarily emphasize schema information while providing limited database content [15]. Due to this imbalance, existing approaches [16–19] have not fully exploited the potential of database content in improving semantic comprehension and relational reasoning. Furthermore, the inherent ambiguities of natural language further exacerbates the difficulty of entity linking tasks. For instance, a discrepancy between "usa" in the database and "United States of America" in the query hinders precise entity matching. Therefore, despite the marked advancements facilitated by LLMs, [3,20] identifies that effectively addressing schema redundancy, limited utilization of database content, and natural language ambiguity remains an open and significant challenge.

In this paper, we propose SRSPSQL, a novel dual-stage Text-to-SQL generation framework that effectively addresses natural language ambiguity and schema redundancy by systematically leveraging dataset

* Corresponding author.
E-mail addresses: Leijj@cqupt.edu.cn (J. Lei), s230231067@stu.cqupt.edu.cn (Z. Li), ywang@cqupt.edu.cn (Y. Wang).

https://doi.org/10.1016/j.infsof.2026.108064
Received 14 August 2025; Received in revised form 29 December 2025; Accepted 2 February 2026
Available online 9 February 2026
0950-5849/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

content information. Specifically, SRSPSQL designs a question semantic reconstruction mechanism that combines the deep contextual understanding of LLMs with the concrete database schema to transform ambiguous user queries into precise, schema-aligned representations. This process not only mitigates ambiguity but also enriches prompt inputs with detailed database context. Based on this, an innovative dual-stage SQL generation paradigm is designed for SQL generation. The first stage generates initial SQL queries via a full-schema prompt constructed from masked question similarity-based example retrieval and database content-augmented schema representations. The second stage further refines generation by applying structure-aware example selection and dynamic schema pruning to create a concise-schema prompt that selectively focuses on critical schema components. This carefully designed dual-stage SQL generation framework significantly enhances query accuracy and computational efficiency, while demonstrating the effectiveness of explicitly decoupling semantic reconstruction from progressive SQL generation in complex Text-to-SQL scenarios. Our main contributions are summarized as follows:

- We propose SRSPSQL, a novel dual-stage framework that synergistically combines semantic question reconstruction and dynamic schema pruning to address natural language ambiguities and schema redundancy, thereby enhancing the SQL generation.
- We design a database-aware semantic reconstruction mechanism that jointly leverages LLMs' generalization capabilities and concrete database instances to enhance understanding of database structures and align questions with the schema.
- We propose a two-stage schema optimization for SQL generation. In the first stage, semantic integrity is ensured through full-schema prompting and content-augmented representations, while the second stage optimizes the generation process via dynamic schema pruning and structure-aware example selection. It significantly improves the accuracy of complex queries while reducing computational overhead.
- Extensive experiments on the Spider and Bird demonstrate that SRSPSQL outperforms existing methods in terms of generalization and compatibility. Ablation studies further confirm the individual and synergistic effectiveness of its semantic reconstruction and schema pruning components.

The remainder of the paper is organized as follows: Section 2 presents the related work. Section 3 details the proposed approach. Section 4 presents and analyzes the experimental results. We conclude this paper in Section 5.

2. Related work

With the rapid advancement and widespread deployment of LLMs in natural language processing, the Text-to-SQL task achieves substantial performance improvements. LLMs demonstrate strong zero-shot reasoning and cross-domain generalization capabilities in translating natural language questions into SQL [4,5]. ACT-SQL [21] adopts automatically generated chain-of-thought (CoT) reasoning to enhance logical inference. DIN-SQL [10] decomposes Text-to-SQL task into sub-modules such as schema linking, query classification, SQL generation, and self-correction, thereby improving SQL generation robustness. Moreover, some studies explore the ICL capabilities of LLMs. Nan et al. [22] systematically investigate the impact of example selection, instruction formats, and schema organization on model performance, while DAIL-SQL [11] optimizes prompt design by refining question representation, example selection, and organization. Despite these advances, most existing methods are heavily dependent on detailed schema representations, resulting in schema redundancy and excessive token consumption, which limits their efficiency and scalability. To address these issues, schema linking mechanisms [23–25] have been proposed to identify and retain only semantically relevant tables and columns. However, these methods adopt rule-based processing and shallow semantic analysis, limiting their ability to capture deep context in complex queries and failing to effectively resolve schema redundancy.

Moreover, mainstream benchmarks such as Spider [14] and WikiSQL [26] focus primarily on schema design and provide limited database content [15]. The distribution mismatch between benchmark datasets and real-world data leads to existing methods overly focusing on database schema, thereby ignoring the importance of actual data content in enhancing semantic understanding and relational reasoning. To address this limitation, graph-based methods [27,28] model semantic relationships between questions and schema to improve cross-database generalization. However, these methods do not incorporate reasoning over the actual database content. Similarly, prompt-based techniques [10,11,29,30] focus predominantly on schema information with little attention to database content. Recently, the introduction of the BIRD [15] highlights the significance of content. Inspired by this, Shen et al. [31] propose ASTRES, which utilizes retrieval-augmented generation (RAG) to incorporate column values into prompts, while Wang et al. [32] introduce MAC-SQL, employing a multi-agent collaborative framework to include randomly sampled values. Nevertheless, these methods typically employ arbitrary sampling or simple concatenation strategies, limiting their ability to fully leverage the complementary semantic relationships between schema and content. Beyond the constraints inherent to datasets, a more fundamental concern in Text-to-SQL evaluation is data contamination and model memorization. Ranaldi et al. [33] highlight that the integrity of performance assessment is critically compromised when benchmark data inadvertently contaminates training sets. Rigorous experimental protocols are essential to distinguish true model generalization from memorization. In addition, to systematically dissect the strengths and common limitations of existing methods, we review and compare 14 representative Text-to-SQL approaches in Section 4. Following the classification frameworks from [3,20], we explicitly categorize them into two main paradigms: ICL-based [10,11,32,34–39] and supervised fine-tuning(SFT)-based [25,40–43].

Despite significant advances in LLM-based Text-to-SQL systems, they still struggle with schema redundancy, inadequate database content utilization, and unresolved linguistic ambiguities. Hence, this paper proposes SRSPSQL, a dual-stage Text-to-SQL framework that combines question semantic rewriting and dynamic schema pruning to generate schema-aligned, content-enriched queries and prune irrelevant schema elements, effectively reducing ambiguity, alleviating redundancy, and enhancing scalability.

3. Method

This section details a dual-stage Text-to-SQL framework that explicitly decouples semantic question reconstruction from SQL generation.

3.1. Overview

Fig. 1 illustrates the execution process of the proposed SRSPSQL framework, which systematically enhances SQL query generation through three tightly integrated stages: (1) The semantic reconstruction phase improves the model's ability to interpret user intent and accurately identify relevant data rows by integrating related database content to optimize the semantic representation of natural language questions; (2) The initial SQL generation phase constructs a comprehensive full-schema prompt by combining the rewritten queries, semantically aligned examples, and content-augmented database schema, enabling the LLM to generate preliminary SQL queries; (3) The final SQL generation phase parses the preliminary queries to extract their structural skeletons and relevant schema elements. Based on this, SRSPSQL performs dynamic example filtering and schema pruning to construct a concise and effective prompt, which the LLM then uses to generate the final optimized SQL queries. By integrating content-aware semantic reconstruction with dynamic schema pruning, SRSPSQL framework effectively enhances both the accuracy and efficiency of SQL generation.

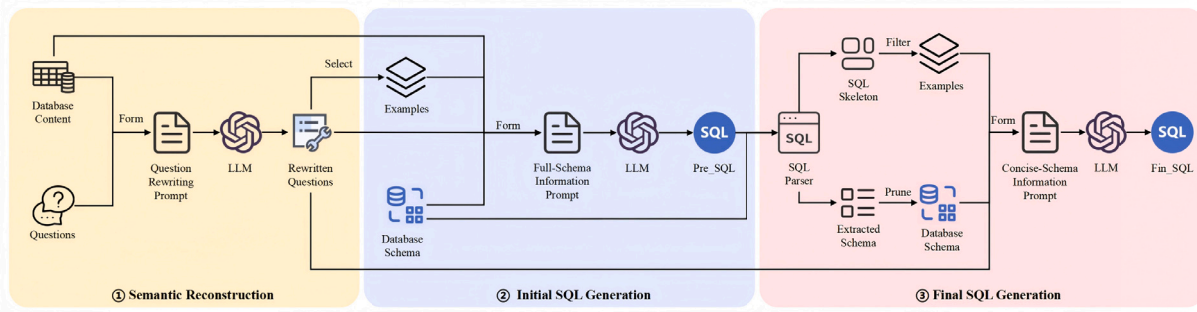


Fig. 1. The execution process of SRSPSQL.

```
/*Instructions: ${INSTRUCTION}*/

/*Here are some basic examples:*/
/*Ensure that the rewritten questions preserve the integrity of the original
semantics, point to specific database objects, and eliminate ambiguous references
and expressions.*/
${EXAMPLE1}
/*If you deem the user's question sufficiently clear, you may return the original
text without rewriting it. */
${EXAMPLE2}

/*Here is the input:*/
/*Given the initial value from Table:*/
${RELEVANT_DATABASE_CONTENT}

/*Rewrite the following question: ${QUESTION}*/
/*Results:*/
```

Listing 1: Semantic reconstruction prompt template, including instruction, example, and input.

3.2. Semantic reconstruction for question rewriting

To eliminate ambiguity in natural language questions, SRSPSQL designs a semantic reconstruction mechanism. As shown in Listing 1, SRSPSQL leverages the LLMs' summarization and generalization capabilities, effectively integrating database content information to rewrite natural language questions, as:

$$Q_{\text{rew}} = \mathcal{M}_{\text{LLM}}\Big(\underbrace{\mathcal{P}_{\text{rew}}\big(\text{Inst},\ \mathcal{E},\ \mathcal{I}(R_{\text{top}_k}, Q_{\text{orig}})\big)}_{\text{question rewriting prompt}}\Big) \tag{1}$$

where Q_{rew} denotes the rewritten query (output), $\mathcal{M}_{\text{LLM}}$ represents the LLM function, $\mathcal{P}_{\text{rew}}$ is the prompt composition function, Inst contains the task instructions (natural language), $\mathcal{E}$ is the example set, $\mathcal{I}$ denotes the constructed input function, R_{top_k} denotes the top-k most relevant data rows selected based on similarity, and Q_{orig} is the original natural language question. Specifically, the question rewriting based on semantic reconstruction is achieved through carefully designed prompts combining instruction, example, and input.

Instruction. To formalize the semantic reconstruction process, we construct an instruction prompt with the following specification: "As a professional natural language question optimizer, rewrite natural language questions based on the provided information. Ensure that the rewritten questions preserve the integrity of the original semantics, clearly point to specific database objects, and eliminate ambiguous references and expressions. If you deem the natural language question sufficiently clear, you may return the original text without rewriting it".

Example. We design two examples to guide the LLMs in reconstructing the semantics of the questions. As shown in Listing 2, example 1 includes three carefully constructed instruction-response pairs, each designed to guide the model in precisely locating relevant database components across three hierarchical levels, namely tables, fields, and data, effectively mitigating potential ambiguities in query interpretation. Example 2 aims to establish a safeguard designed to prevent the model from rewriting queries that are already sufficiently clear, thus maintaining both the semantic integrity and accuracy of the original query and avoiding unnecessary or detrimental modifications.

Input. We take the original natural language question and related database content information as inputs to the LLM. Specifically, the database content information R_{top_k} is extracted by semantic-aware dynamic table selection and database content filtering.

(1) Semantic-aware dynamic table selection. Given a database table set $\mathcal{T} = \{t_1, t_2, \ldots, t_m\}$, we first perform table structure vectorization for each table $t_i \in \mathcal{T}$ by concatenating the table name and all column names into the string S_i, and then obtaining its structure embedding via the pre-trained BERT model [44] as $v_{S_i} = \text{BERT}(S_i) \in \mathbb{R}^{768}$. In parallel, the natural language question Q_{orig} is encoded into the same vector space to capture query intent, denoted as $v_Q^{\text{table}} = \text{BERT}(Q_{\text{orig}}) \in \mathbb{R}^{768}$.

Unlike traditional keyword-matching techniques, SRSPSQL introduces deep semantic vector space alignment to model implicit relationships, enabling the computation of fine-grained semantic similarity scores between v_{S_i} and v_Q^{table}:

$$\text{sim}_{\cos}(v_{S_i}, v_Q^{\text{table}}) = \frac{v_{S_i} \cdot v_Q^{\text{table}}}{\|v_{S_i}\| \|v_Q^{\text{table}}\|} \tag{2}$$

where $\|\cdot\|$ denotes the magnitude (or norm) of a vector.

Based on similarity scores, SRSPSQL identifies the top-N tables in $\mathcal{T}$ that exhibit the highest semantic alignment, as:

$$\mathcal{T}_{\text{sel}} = \arg \max_{\mathcal{T}' \subseteq \mathcal{T}, |\mathcal{T}'| = N} \sum_{t_i \in \mathcal{T}'} sim_{cos}(v_{S_i}, v_Q^{\text{table}}), \tag{3}$$

```
/*Example 1*/
/*Rewrite the following question:Find the number of distinct species of domestic
animals.*/
/*Results:Find the number of distinct pet types in the table "Pets".*/
/*Rewrite the following question:Show the name and the publish year of the song by
the youngest vocalists.*/
/*Results:Show the name and song_release_year the of song by the youngest singer.*/
/*Rewrite the following question:What is the count of the vehicle models produced in
the United States?*/
/*Results:What is the count of the car models produced in the "usa"?*/

/*Example 2*/
/*Rewrite the following question:How many singers do we have?*/
/*Results:How many singers do we have?*/
/*Rewrite the following question:What are the names, themes, and number of singers
every concert?*/
/*Results:What are the names, themes, and number of singers for every concert?*/
```

Listing 2: Example of two types of reconstruction questions.

where T' represents a subset of $\mathcal{T}$, and $|\cdot|$ denotes the size of a set. Statistical analysis shows that over 97% of benchmark queries involve no more than three tables. Empirical tests further confirm that compared to $N = 3$, setting $N = 4$ results in a 12%–19% increase in token overhead while yielding only a 0.2–0.3% improvement in accuracy. Therefore, to balance prompt length constraints with the complexity of multi-table queries, we set N to 3.

(2) Database content filtering base on semantic alignment. Semantic similarity is employed to select the top-k data rows most relevant to Q_{orig}, designated as database content information, which serves to improve SQL generation accuracy and efficiency while suppressing noise and lowering the computational demands placed on LLMs.

Let R denote the set of rows from the tables in $\mathcal{T}_{\text{sel}}$. We concatenate the fields of each row from every table in $\mathcal{T}_{\text{sel}}$ into a concise textual description, denoted as r_j. We adopt the pre-trained GloVe model [45] to compute sequence embeddings for $r_j \in R$ and Q_{orig}, denoted as $v_{r_j} = \frac{1}{|r_j|}\sum_{\omega\in r_j}\text{GloVe}(\omega)$ and $v_Q^{\text{row}} = \frac{1}{|Q_{\text{orig}}|}\sum_{\omega\in Q_{\text{orig}}}\text{GloVe}(\omega)$. To measure their semantic relevance, we employ cosine similarity to calculate the alignment score between v_{r_j} and v_Q^{row}:

$$\text{sim}_{\cos}(v_{r_j}, v_Q^{\text{row}}) = \frac{v_{r_j}\cdot v_Q^{\text{row}}}{\|v_{r_j}\|\|v_Q^{\text{row}}\|} \tag{4}$$

Based on this, the top-k rows that exhibit the highest similarity to Q_{orig} can be obtained as:

$$R_{\text{top}_k} = \arg\max_{R'\subseteq R, |R'|=K} \sum_{r_j\in R'} \text{sim}_{\cos}(v_{r_j}, v_Q^{\text{row}}), \tag{5}$$

where R' is a subset of R. To optimize the trade-off between data relevance and prompt efficiency, we empirically set $K = 5$.

3.3. Full-schema prompt for initial SQL generation

To further enhance LLMs' understanding of database schemas, SR-SPSQL constructs a comprehensive full-schema prompt that integrates relevant examples with schema information. As illustrated in Listing 3, the full-schema prompt serves to guide the generation of schema-aware initial SQL, which is formally expressed as:

$$\text{SQL}_{\text{init}} = \mathcal{M}_{\text{LLM}}\Big(\mathcal{P}_{\text{init}}\big(\underbrace{\mathcal{E}(Q_{\text{rew}})}_{\text{examples}}, \underbrace{\mathcal{S}(D)}_{\text{schema}}, \underbrace{Q_{\text{rew}}}_{\text{question}}\big)\Big) \tag{6}$$

where $\mathcal{P}_{\text{init}}$ denotes the prompt operation for initial SQL generation, $\mathcal{E}(Q_{\text{rew}})$ represents the few-shot examples retrieved based on masked question similarity, $\mathcal{S}(D)$ is the full database schema including tables, columns, and constraints.

(1) Example retrieval based on masked question similarity. Inspired by the masked question similarity selection method (MQS_S) [46], we replace table names, column names, and values in Q_{rew} and each candidate question $q_i \in Q_c = \{q_1, \ldots, q_n\}$ with [MASK] tokens, denoted as Q_{masked} and $q_{i,\text{masked}}$. This replacement eliminates the negative influence of domain-specific information.

To quantify the similarity between these masked questions, the BERT model [44] is employed to encode Q_{masked} and $q_{i,\text{masked}}$ separately, generating their corresponding vector representations as:

$$\mathbf{v}_Q = \text{BERT}(Q_{\text{masked}}), \tag{7}$$

$$\mathbf{v}_{q_i} = \text{BERT}(q_{i,\text{masked}}), \tag{8}$$

where $\mathbf{v}_Q, \mathbf{v}_{q_i} \in \mathbb{R}^{768}$ are L_2-normalized embeddings.

The semantic alignment between $\mathbf{v}_Q$ and $\mathbf{v}_{q_i}$ is quantified using cosine similarity, which effectively captures both lexical and syntactic relationships, as:

$$\text{sim}_{cos}(\mathbf{v}_Q, \mathbf{v}_{q_i}) = \frac{\mathbf{v}_Q\cdot\mathbf{v}_{q_i}}{\|\mathbf{v}_Q\|\cdot\|\mathbf{v}_{q_i}\|} \tag{9}$$

Moreover, the example retrieval function $\mathcal{E}(Q_{\text{rew}})$ is designed to identify the top-k most relevant examples $\{(Q, q_i)\}$ from the training set $\mathcal{Q}_{\text{train}}$ by:

$$\mathcal{E}(Q_{\text{rew}}) = \underset{\{(Q,q_i)\}\in\mathcal{Q}_{\text{train}}}{\arg\text{top-}k}\ \text{sim}_{cos}(\mathbf{v}_Q, \mathbf{v}_{q_i}). \tag{10}$$

Accordingly, the top-k examples with the highest semantic similarity scores are selected for few-shot prompting. We select examples based on similarity in query intent rather than surface-level vocabulary or table structure. This design enables the model to learn and compare the abstract logical structures between queries, such as operation types, conditional relationships, and aggregation patterns. It effectively addresses the potential issue where examples share similar table structures but differ in query intent, thereby ensuring that the selected examples provide effective contextual information for target SQL generation. These examples are then organized following the DAIL paradigm [11], which dynamically adapts the ordering and composition of few-shot examples based on their semantic relevance to the target query.

(2) Content-augmented schema construction. Given the complete database $D = \{T_1, \ldots, T_m\}$ with schema $\mathcal{S}$ and rewritten query Q_{rew}, we construct an enhanced schema representation: $\mathcal{S}(D) = \langle \mathcal{S}, R'_{\text{top}_k}\rangle$ where

```
/*Instructions: Generate correct and efficient SQL queries based on examples and
database schema, ensuring minimized execution time.*/

/*Some example questions and corresponding SQL queries are provided based on
similar problems:*/
${EXAMPLE}

/*Given the following database schema:*/
${ENHANCED_DATABASE_SCHEMA}

/*Answer the following: ${QUESTION}*/
SELECT
```

Listing 3: Full-Schema prompt template, including instructions, relevant examples, enhanced database schema, and query question.

```
/* Given the following database schema:*/
CREATE TABLE "continents" (
"ContId" INTEGER PRIMARY KEY,
"Continent" TEXT
);

Given the following information for the first five rows:
INSERT INTO continents ( ContId , Continent ) VALUES ( '5' , 'australia' ) ,
( '2' , 'europe' ),
( '3' , 'asia' ),
( '4' , 'africa' ),
( '1' , 'america' );
```

Listing 4: Example of enhanced schema information.

R'_{top_k} contains the k most relevant rows from tables $T \in \mathcal{D}$, selected according to the similarity metric in (5). As shown in Listing 4, this augmented schema provides the model with richer contextual information for SQL generation. Note that the content-augmented schema construction satisfies:

$$I(S(D); Q_{\text{rew}}) \geq I(S; Q_{\text{rew}}) + \alpha \cdot I(R'_{\text{top}_k}; Q_{\text{rew}}) \tag{11}$$

where $I(\cdot;\cdot)$ denotes mutual information function. By incorporating the relevant rows R'_{top_k}, $S(D)$ contains more information related to Q_{rew} than S, with an information gain of at least $\alpha \cdot I(R'_{\text{top}_k}; Q_{\text{rew}})$.

Accordingly, SRSPSQL leverages this augmented information to strengthen the model's comprehension of the database schema and, by selecting rows most relevant to the query, substantially reduces value-related errors, thereby improving the accuracy of SQL generation.

3.4. Concise-schema prompt for final SQL generation

To further enhance SQL accuracy, we adopt a two-stage generation framework, in which the second stage formulates an optimized prompt $\mathcal{P}_{\text{final}}$ by integrating syntactically relevant examples ($\mathcal{E}_{\text{filter}}$), the pruned schema ($\mathcal{S}_{\text{pruned}}$), and Q_{rew}. The final SQL query is then produced by:

$$\text{SQL}_{\text{final}} = \mathcal{M}_{\text{LLM}}\Big(\mathcal{P}_{\text{final}}\big(\underbrace{\mathcal{E}_{\text{filter}}}_{\text{examples}}, \underbrace{\mathcal{S}_{\text{pruned}}}_{\text{schema}}, \underbrace{Q_{\text{rew}}}_{\text{question}}\big)\Big), \tag{12}$$

where $\mathcal{M}_{\text{LLM}}$ denotes the inference function of the LLM. $\mathcal{P}_{\text{final}}$ is constructed following Algorithm 1, encompassing three key processes: SQL parsing for skeleton and schema extraction, structure-aware example filtering, and schema pruning.

(1) SQL parsing for skeleton and schema extraction. As shown in Fig. 2, the parser converts the SQL_{init} and the full schema $\mathcal{S}_{\text{full}}$ into normalized forms. The SQL_{init} and $\mathcal{S}_{\text{full}}$ are first case-normalized and syntactically cleaned: $Q_{\text{norm}} = \text{lower}(Q_{\text{init}}) \circ \phi_{\text{clean}}$, where ϕ_{clean} removes redundant symbols, and $\mathcal{S}_{\text{norm}} = \{\text{lower}(t), \text{lower}(c) \mid t \in \mathcal{T}, c \in t.\text{columns}\}$, where $\mathcal{T}$ and $\mathcal{C}$ denote the sets of tables and columns, respectively.

Algorithm 1: Concise-Schema Prompt Generation Algorithm

Input: SQL_{init}: Initially generated SQL query
$\mathcal{S}_{\text{full}}$: Full database schema
$\mathcal{D}_{\text{SQL}}$: Candidate SQL pool
Q_{rew}: Rewritten question
Output: $\mathcal{P}_{\text{final}}$: Final prompt for SQL generation
1: $(Q_{\text{ske}}, \mathcal{S}_{\text{used}}) \leftarrow$ **ParSQL**$(\text{SQL}_{\text{init}}, \mathcal{S}_{\text{full}})$;
2: $\mathcal{E}_{\text{filter}} \leftarrow$ **FiltExa**$(Q_{\text{ske}}, \mathcal{D}_{\text{SQL}})$;
3: $\mathcal{S}_{\text{pruned}} \leftarrow$ **Prunesche**$(\mathcal{S}_{\text{full}}, \mathcal{S}_{\text{used}})$;
4: $\mathcal{P}_{\text{final}} \leftarrow$ **ConstructPrompt**$(\mathcal{E}_{\text{filter}}, \mathcal{S}_{\text{pruned}}, Q_{\text{rew}})$;
5: Return $\mathcal{P}_{\text{final}}$

To ensure the integrity of SQL keywords and clause structure, skeleton extraction replaces all table names $\mathcal{T}$, column names $\mathcal{C}$, and literal values $\mathcal{V}$ with generic placeholders, as:

$$Q_{\text{ske}} = \psi(Q_{\text{norm}}, \{\mathcal{T}, \mathcal{C}, \mathcal{V}\}, \text{“_”}), \tag{13}$$

As shown in Fig. 2, the SQL query `SELECT ContId FROM continents` is transformed into `SELECT _ FROM _`.

In parallel, schema extraction is employed to identify relevant schema elements:

$$\mathcal{S}_{\text{used}} = \langle \mathcal{T}_{\text{used}}, \mathcal{C}_{\text{used}}, \mathcal{K}_{\text{pk/fk}} \rangle, \tag{14}$$

where $\mathcal{T}_{\text{used}} = \{t \mid \exists c \in t : c \in Q_{\text{norm}}\}$ are the tables accessed by the query, $\mathcal{C}_{\text{used}} = \{c \mid c \in Q_{\text{norm}}\}$ are the referenced columns, and $\mathcal{K}_{\text{pk/fk}}$ are the associated primary/foreign key constraints extracted from $\mathcal{S}_{\text{norm}}$. Hence, the SQL parsing process is expressed as:

$$(Q_{\text{ske}}, \mathcal{S}_{\text{used}}) = \textbf{ParSQL}(\text{SQL}_{\text{init}}, \mathcal{S}_{\text{full}}), \tag{15}$$

where $(Q_{\text{ske}}, \mathcal{S}_{\text{used}})$ jointly capture the query's structural pattern and its schema dependencies, thus enabling structure-aware SQL refinement.

(2) Structure-aware example filtering. SRSPSQL retrieves structurally relevant examples based on Q_{ske} and $\mathcal{D}_{\text{SQL}}$. Specifically, it converts Q_{ske} and each candidate SQL skeleton $S_{c_i} \in \mathcal{D}_{\text{SQL}}$ into abstract

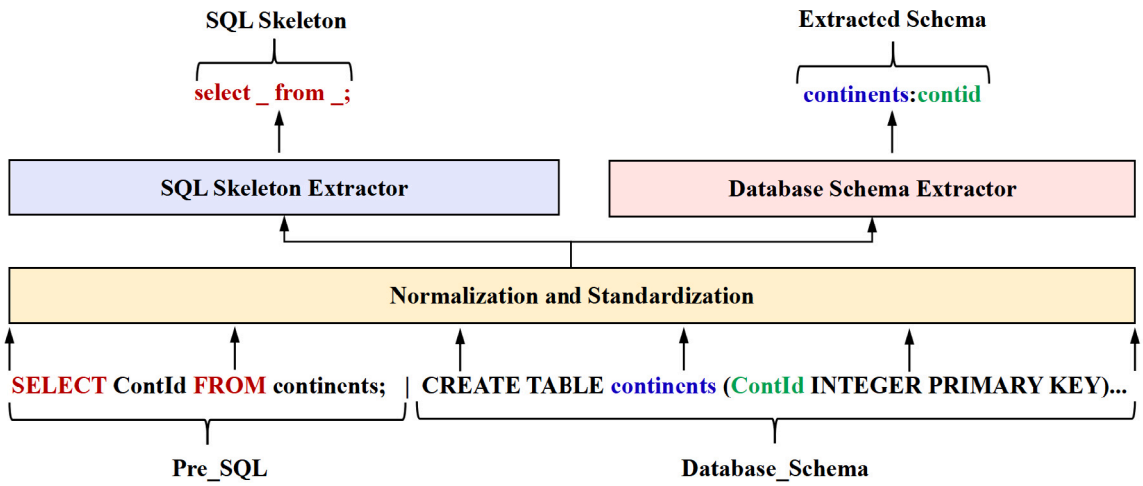


Fig. 2. The process of parsing the initially generated SQL.

```
/*Given the following database schema:*/
CREATE TABLE "cars_data " (...);
Given the following information for the first five rows:
INSERT INTO cars_data (Id, MPG, Cylinders, Edispl, Horsepower, Weight,Year) VALUES (...);

CREATE TABLE "continents" (...);
Given the following information for the first five rows:
INSERT INTO continents (ContId,Continent) VALUES (...);
```

Listing 5: Example of schema pruning.

syntax trees (ASTs) A_{query} and A_{cand_i}, respectively. The similarity score of Q_{ske} and S_{c_i} is computed as:

$$\text{sim}(Q_{\text{ske}}, S_{\text{c}_i}) = 1 - \frac{\text{APTED}(A_{\text{query}}, A_{\text{cand}_i})}{\max(|A_{\text{query}}|, |A_{\text{cand}_i}|)}, \tag{16}$$

where $|A_{\text{query}}|$ and $|A_{\text{cand}_i}|$ denote the number of nodes in the corresponding ASTs, and APTED$(\cdot)$ represents the tree edit distance between two ASTs.

Subsequently, candidate SQL skeletons are ranked according to their similarity scores:

$$R_s = \underset{S_{\text{c}_i} \in D_{\text{SQL}}}{\arg\ \text{sort}} \left(\text{sim}(Q_{\text{ske}}, S_{\text{c}_i})\right)_{\downarrow}, \tag{17}$$

where R_s orders candidates from highest to lowest similarity. Based on R_s, the top-k examples can be obtained as:

$$\mathcal{E}_{\text{filter}} = \textbf{FiltExa}(Q_{\text{ske}}, \mathcal{D}_{\text{SQL}}) = \{S_{\text{c}_i} \mid i \in R_s[1:K]\} \tag{18}$$

This approach ensures that the selected examples exhibit structural similarity to the target query (e.g., comparable JOIN patterns or nested conditions), thereby enhancing the relevance and effectiveness of the few-shot examples.

(3) Schema pruning. To minimize prompt redundancy, we prune the full schema $\mathcal{S}_{\text{full}}$ by retaining only the tables and related attributes referenced in the extracted schema information $\mathcal{S}_{\text{used}}$. The pruned schema is formally defined as:

$$\mathcal{S}_{\text{pruned}} = \textbf{Prunesche}(\mathcal{S}_{\text{full}}, \mathcal{S}_{\text{used}}). \tag{19}$$

Compared to column-level linking, table-level pruning offers enhanced robustness, as excessive filtering at the column level may exclude pertinent contextual information, thereby impairing the LLM's comprehension and consequently degrading the quality of SQL generation. Moreover, schema pruning effectively reduces redundant information within the prompt, with empirical evidence demonstrating a reduction in token consumption by up to 44%.

4. Experiments

4.1. Experimental setup

Datasets. We evaluate SRSPSQL on Spider [14], Bird [15], and two perturbed datasets, Spider-Syn [47] and Spider-Realistic [48].

- Spider [14] is a large-scale cross-domain Text-to-SQL dataset. The training set contains 8659 examples, the development set includes 1034 examples, and the test set consists of 2147 examples, covering 200 multi-table databases.
- Bird [15], developed by Alibaba DAMO Academy, is a new dataset for large-scale real-world databases. It contains 95 large-scale databases with high-quality Text-to-SQL pairs, featuring a total storage volume of 33.4 GB across 37 professional domains.
- Spider-Syn [47] is a robustness dataset derived from the Spider. The dataset is generated by manually replacing schema-related terms (e.g., table and column names) in natural language questions with synonyms. In total, 5672 questions are revised through 273 single-word and 189 multi-word synonym substitutions, averaging approximately one modification per question.
- Spider-Realistic [48] is a challenging evaluation dataset derived from the Spider. It consists of 508 examples selected from the Spider-dev set. The key modification involves removing explicit column names from the natural language questions while keeping SQL queries unchanged.

Evaluation Metrics. To ensure a fair comparison, we follow previous studies and use exact-set-match accuracy (EM) and execution accuracy (EX) to evaluate the effectiveness of our SRSPSQL. EM splits the predicted SQL statement and the ground truth SQL statement into sequences of SQL keywords and checks whether their sets of SQL keywords are identical. EX compares the execution results of the predicted SQL statement and the ground truth SQL statement on the corresponding database.

Table 1
EX (%) comparison of different Text-to-SQL methods on Spider dev and test dataset. The Size column represents the number of parameters of the model, where UNK represents unknown size and B represents billions.

Method	Type	Model	Size	EX(Dev)	EX(Test)
DIN-SQL (Pourreza and Rafiei, 2023)[a] [10]	ICL	Deepseek-v3-0324	660B	83.5	82.4
DAIL-SQL (Gao et al. 2024)[a] [11]	ICL	Deepseek-v3-0324	660B	80.9	79.9
PTD-SQL (Luo et al. 2024) [34]	ICL	GPT-4	UNK	85.7	–
TA-SQL (Qu et al. 2024) [35]	ICL	GPT-4	UNK	85.0	–
MAC-SQL (Wang et al. 2025)[a] [32]	ICL	Deepseek-v3-0324	660B	80.0	77.9
MAC-SQL (Wang et al. 2025) [32]	ICL	GPT-3.5-Turbo	UNK	80.6	75.5
Solid-SQL (Liu et al. 2025) [36]	ICL	Llama3	70B	82.1	–
Solid-SQL (Liu et al. 2025) [36]	ICL	GPT-4o-mini	UNK	79.9	–
COGSQL (Yuan et al. 2025) [37]	ICL	GPT-4o-mini	UNK	84.2	83.8
Gen-SQL (Shi et al. 2025) [38]	ICL	GPT-4	UNK	85.6	–
DCG-SQL (Lee et al. 2025) [39]	ICL	GPT-3.5-Turbo	UNK	83.3	–
RESDSQL (Li et al. 2023b) [25]	SFT	T5	3B	84.1	79.9
SENSE (Yang et al. 2024b) [40]	SFT	CodeLLaMA	7B	83.2	83.5
CodeS (Li et al. 2024a) [41]	SFT	StarCoder	15B	84.9	–
CodeS (Li et al. 2024a) [41]	SFT	StarCoder	7B	85.4	–
DTS-SQL (Pourreza and Rafiei, 2024) [42]	SFT	DeepSeek	7B	–	84.4
MSc-SQL (Gorti et al. 2025) [43]	SFT	Mistral-v0.3	9B	–	84.7
SRSPSQL(Ours)	ICL	Deepseek-v3-0324	660B	85.4	85.6
SRSPSQL(Ours)	ICL	Qwen2.5-Coder	14B	86.1	84.8
SRSPSQL(Ours)	ICL	Qwen2.5-Coder	32B	**86.4**	**85.8**

[a] Means reproduction results with released code by ourselves.

Baseline. We compare our SRSPSQL method with multiple baseline methods that ICL-based [10,11,32,34–39], and SFT-based [25,40–43].

1. DIN-SQL [10] employs CoT prompt to decompose the Text-to-SQL task into subproblems: schema linking, query classification and decomposition, SQL generation, and self-correction, enhancing the model's capability to handle complex queries.
2. DAIL-SQL [11] employs various example selection strategies to optimize example filtering and structurally organizes examples to improve the quality of SQL generation.
3. RESDSQL [25] employs ranking-enhanced encoding and skeleton-aware decoding to decouple schema linking from SQL skeleton parsing, thereby alleviating the complexity of Text-to-SQL parsing.
4. MAC-SQL [32] adopts a multi-agent framework: a core decomposer for Text-to-SQL with CoT reasoning, with auxiliary agents handling sub-database extraction and SQL refinement. Agents collaborate flexibly, allowing new tool integration for enhanced parsing.
5. PTD-SQL [34] enhances LLMs Text-to-SQL reasoning by partitioning queries into groups. This enables focused learning on specific problem types through categorical thinking, improving abilities across difficulties and categories.
6. TA-SQL [35] mitigates Text-to-SQL hallucinations via task alignment. This strategy leverages experiences from similar tasks to reduce LLMs' generalization burden, enhancing robustness.
7. Solid-SQL [36] enhances schema-linking via LLM-augmented data and structural similarity-based example retrieval against adversarial perturbations.
8. COGSQL [37] enhances Text-to-SQL by mimicking human cognition through three stages: schema linking, concept-augmented CoT generation, and dual-consistency error correction.
9. Gen-SQL [38] introduces a generate-ground-regenerate framework that enhances schema retrieval accuracy while reducing costs by leveraging LLMs' prior knowledge.
10. DCG-SQL [39] enhances Text-to-SQL via deep contextual schema link graphs, improving demonstration retrieval and SQL generation across LLM scales.
11. SENSE [40] achieves Text-to-SQL domain generalization by integrating strong and weak model data through preference learning and instruction tuning, adapting open LLMs into the SENSE framework.
12. CodeS [41] introduces open-source models for Text-to-SQL, specialized via pretraining, prompt design, and bidirectional augmentation to address schema linking and domain adaptation.
13. DTS-SQL [42] introduces a two-stage fine-tuning strategy for Text-to-SQL, which decomposes the task to better align open-source models with their proprietary counterparts.
14. MSc-SQL [43] develops compact and efficient open-source Text-to-SQL models, introducing a mechanism to evaluate multiple SQL candidates using metadata.

Implementation Details. We conduct experiments using three LLMs with various parameter sizes. Specifically, Deepseek-v3-0324 is a 660-billion-parameter model from the Deepseek series, known for its strong text comprehension and code generation capabilities. The Qwen2.5-coder-32B and Qwen2.5-coder-14B models, belonging to the Qwen2.5 series, contain 32 billion and 14 billion parameters, respectively, and excel at understanding structured data and generating structured outputs. All models are accessed via API, with the temperature parameter fixed at 0.0. In the prompt design, both the number of relevant examples and the number of relevant database rows are set to five. Regarding computational efficiency, the average time cost for generating one SQL query is 7.9 s using Deepseek-v3-0324 and 6.6 s using the Qwen2.5-Coder series models. The time spent on extracting relevant database rows is 0.73 s.

4.2. Experimental results

We test the performance of SRSPSQL across various benchmarks and compare it with baseline approaches.

As evidenced by Table 1, SRSPSQL demonstrates state-of-the-art performance on the Spider across multiple LLM configurations. When implemented with the Qwen2.5-Coder-32B model, SRSPSQL achieves EX of 86.4% on the development set and 85.8% on the test set, outperforming all existing ICL and SFT baselines. Notably, even with the more compact Qwen2.5-Coder-14B variant, SRSPSQL maintains robust performance at 86.1% (dev) and 84.8% (test), outperforming both specialized SFT approaches and GPT-4-powered ICL methods. Comparative analysis reveals that SRSPSQL achieves consistent improvements of +3.9% (dev) and +5.5% (test) over competing prompt-based methods (DIN-SQL [10], DAIL-SQL [11], MAC-SQL [32]) when using identical Deepseek-v3-0324 model parameters. This performance advantage is attained without resorting to the resource-intensive strategies employed

Table 2
EX (%) comparison of different Text-to-SQL methods on perturbed datasets Spider-Syn and Spider-Realistic. The Size column represents the number of parameters of the model, where UNK represents unknown size and B represents billions.

Method	Type	Model	Size	EX(Syn)	EX(Realistic)
DIN-SQL (Pourreza and Rafiei, 2023)[a] [10]	ICL	Deepseek-v3-0324	660B	76.6	77.8
DAIL-SQL (Gao et al. 2024)[a] [11]	ICL	Deepseek-v3-0324	660B	72.1	77.0
PTD-SQL (Luo et al. 2024) [34]	ICL	ChatGPT	UNK	–	72.2
TA-SQL (Qu et al. 2024) [35]	ICL	GPT-4	UNK	–	79.5
MAC-SQL (Wang et al. 2025)[a] [32]	ICL	Deepseek-v3-0324	660B	69.1	78.9
Solid-SQL (Liu et al. 2025) [36]	ICL	Llama3	70B	74.6	77.1
Solid-SQL (Liu et al. 2025) [36]	ICL	GPT-4o-mini	UNK	74.6	76.7
COGSQL (Yuan et al. 2025) [37]	ICL	GPT-4o-mini	UNK	76.0	81.5
DCG-SQL (Lee et al. 2025) [39]	ICL	GPT-3.5-Turbo	UNK	74.2	79.0
RESDSQL (Li et al. 2023b) [25]	SFT	T5	3B	76.9	81.9
SENSE (Yang et al. 2024b) [40]	SFT	CodeLLaMA	7B	72.6	82.7
CodeS (Li et al. 2024a) [41]	SFT	StarCoder	15B	77.0	83.1
CodeS (Li et al. 2024a) [41]	SFT	StarCoder	7B	76.9	82.9
SRSPSQL(Ours)	ICL	Deepseek-v3-0324	660B	76.7	82.3
SRSPSQL(Ours)	ICL	Qwen2.5-Coder	14B	75.5	82.5
SRSPSQL(Ours)	ICL	Qwen2.5-Coder	32B	**78.0**	**83.3**

[a] Means reproduction results with released code by ourselves.

Table 3
EX (%) comparison of different Text-to-SQL methods on Bird dev dataset. The Size column represents the number of parameters of the model, where UNK represents unknown size and B represents billions.

Method	Type	Model	Size	EX(Dev)
DIN-SQL (Pourreza and Rafiei, 2023)[a] [10]	ICL	Deepseek-v3-0324	660B	55.28
DIN-SQL (Pourreza and Rafiei, 2023) [10]	ICL	GPT-4	UNK	50.72
DAIL-SQL (Gao et al. 2024)[a] [11]	ICL	Deepseek-v3-0324	660B	54.89
PTD-SQL (Luo et al. 2024) [34]	ICL	GPT-4	UNK	57.0
TA-SQL (Qu et al. 2024) [35]	ICL	GPT-4	UNK	56.19
MAC-SQL (Wang et al. 2025)[a] [32]	ICL	Deepseek-v3-0324	660B	58.60
MAC-SQL (Wang et al. 2025) [32]	ICL	GPT-4	UNK	59.39
Solid-SQL (Liu et al. 2025) [36]	ICL	GPT-4	UNK	58.90
COGSQL (Yuan et al. 2025) [37]	ICL	GPT-4	UNK	59.58
Gen-SQL (Shi et al. 2025) [38]	ICL	GPT-4	UNK	59.80
DCG-SQL (Lee et al. 2025) [39]	ICL	GPT-3.5-Turbo	UNK	49.70
SENSE (Yang et al. 2024b) [40]	SFT	CodeLLaMA	13B	55.48
SENSE (Yang et al. 2024b) [40]	SFT	CodeLLaMA	7B	51.80
CodeS (Li et al. 2024a) [41]	SFT	StarCoder	15B	58.47
CodeS (Li et al. 2024a) [41]	SFT	StarCoder	7B	57.17
DTS-SQL (Pourreza and Rafiei, 2024) [42]	SFT	DeepSeek	7B	55.80
SRSPSQL(Ours)	ICL	Deepseek-v3-0324	660B	**62.13**
SRSPSQL(Ours)	ICL	Qwen2.5-Coder	14B	60.19
SRSPSQL(Ours)	ICL	Qwen2.5-Coder	32B	61.54

[a] Means reproduction results with released code by ourselves.

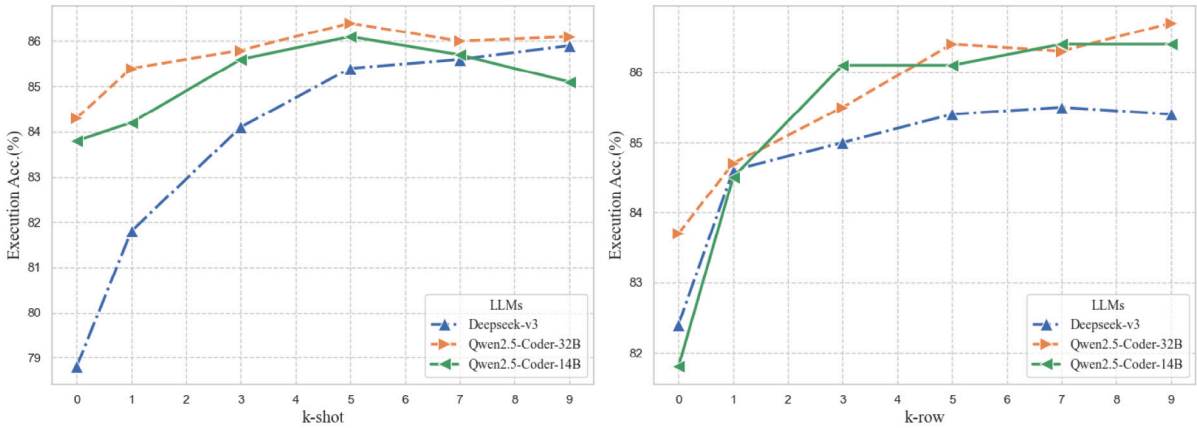
by PTD-SQL [34] and CodeS [41], which rely on either excessively large models (e.g., GPT-4) or computationally expensive fine-tuning strategies. These results confirm the robustness and scalability of SRSPSQL in balancing performance and model size.

As shown in Table 2, SRSPSQL demonstrates robust performance on perturbed datasets designed to simulate real-world challenges. On Spider-Syn, which introduces semantic-based ambiguity, our SRSPSQL achieves an EX score of **78.0%** using Qwen2.5-Coder-32B, outperforming both ICL and SFT baselines, including CodeS [41] (+1%) and RESDSQL [25] (+1.1%). Similarly, on Spider-Realistic, where schema-related terms are deliberately omitted, SRSPSQL attains **83.3%** accuracy, outperforming CodeS (83.1%) and SENSE [40] (82.7%). This performance is achieved without fine-tuning or external data, relying instead on semantic reconstruction to infer user intent beyond lexical matching. The adaptability of SRSPSQL is further demonstrated by consistent performance across model scales: Qwen2.5-Coder-14B (75.5/82.5) achieves accuracy within 2.5% of the 32B variant (78.0/83.3), while Deepseek-v3-0324 (76.7/82.3, 660B) delivers comparable results, confirming the scalability and robustness of SRSPSQL across different LLM capacities.

Table 3 demonstrates the experimental results of SRSPSQL on the Bird. Compared with Spider, Bird exhibits higher complexity and more accurately reflects real-world application scenarios. Despite these challenges, SRSPSQL achieves state-of-the-art performance across both ICL and SFT methods, highlighting its effectiveness under more demanding conditions. Leveraging the Deepseek-v3-0324 model, SRSPSQL attains an EX score of 62.13%, outperforming GPT-4-based ICL methods such as MAC-SQL [32], COGSQL [37], and Gen-SQL [38]. Even with smaller open-source backbones such as Qwen2.5-Coder-14B and Qwen2.5-Coder-32B, it still achieves 60.19% and 61.54%, respectively, demonstrating strong parameter efficiency. In contrast, the optimal SFT approach, CodeS [41] with StarCoder-15B, reaches only 58.47%. Furthermore, compared with the SENSE method [40], which relies on hybrid strong-weak model integration, SRSPSQL consistently maintains high performance across various model scales, demonstrating its scalability and robustness on the more challenging Bird dataset.

4.3. Ablation study

Table 4 presents the ablation results of the SQL generation model based on Deepseek-v3-0324. We analyze the contributions of the proposed semantic reconstruction (SR) and concise-schema prompt generation (CSP) modules. The complete SRSPSQL framework achieves the best performance across all three datasets, with particularly significant gains on Spider-Syn and Spider-Realistic, which involve synonym substitutions and missing column names. Removing SR results in a 3.37%



(a) Few-shot on Spider-Dev (b) Few-row on Spider-Dev

Fig. 3. Few-shot and Few-row evaluation with distinct LLMs on Spider-Dev.

Table 4
The ablation study of SRSPSQL. "w/o SR" refers to removing the semantic reconstruction, "w/o CSP" refers to removing the concise-schema prompt generation.

Method	Spider-Dev		Spider-Syn		Spider-Realistic	
	EX (%)	EM (%)	EX (%)	EM (%)	EX (%)	EM (%)
SRSPSQL	**85.4**	**65.8**	**76.7**	**57.2**	**82.3**	**61.2**
w/o SR	81.4	64.7	73.4	51.4	79.5	60.8
w/o CSP	84.8	65.3	76.4	57.1	81.3	61.0

Table 5
The impact of example quantity in prompts on token overhead and average EX (%).

k-shot	0	1	3	5[a]	7	9
Token (K)	522.5	569.7	664.3	759.6	854.6	950.8
Avg (EX)	82.23	83.80	85.17	**85.97**	85.77	85.70

[a] Means the selected number of parameters.

decrease in average EX and a 2.43% decrease in EM, calculated as the relative performance drops of the SR-ablated model compared to the full SRSPSQL across the Spider, Spider-Syn, and Spider-Realistic. These consistent declines highlight that the SR module effectively reconstructs query semantics, enabling the model to accurately interpret user intent even under challenging conditions such as synonym substitutions and omitted column names, thereby substantially improving SQL generation accuracy. Removing CSP leads to a 0.63% drop in EX and a 0.27% drop in EM, confirming that concise-schema prompt generation, through schema pruning and example selection based on the initially generated SQL, reduces schema redundancy and provides LLMs with cleaner, more focused prompts.

The ablation study validates the effectiveness of both the SR and CSP modules. SR enhances the robustness of semantic interpretation, while CSP optimizes prompt construction by selectively filtering schema information. Each component contributes individually to the overall performance, while their synergistic effect further enhances SRSPSQL's capability to generate robust and generalizable SQL by effectively reducing linguistic ambiguity and schema redundancy.

4.4. Hyper-parameter study

We conduct a systematic analysis on the Spider-Dev to evaluate the SQL generation performance of LLMs with various parameter scales under different prompt configurations. As shown in Fig. 3(a), when using Qwen2.5-Coder as the base model, EX initially increases and subsequently decreases as the number of relevant examples grows, with optimal performance achieved at five examples. This phenomenon can be attributed to the smaller capacity of models like Qwen2.5-Coder (14B/32B), which are more susceptible to prompt noise and information overload beyond a certain point, leading to degraded reasoning focus and accuracy. In contrast, with the Deepseek-v3-0324 model, EX exhibits a clear upward trend, stabilizing after five relevant example pairs, beyond which accuracy gains become marginal. Table 5 details the corresponding token consumption and average EX of the three models on Spider-Dev as the number of examples increases. Consistent with the observed trends, all three models reach peak average EX of 85.97% at five examples. Hence, we set the number of examples to five for ensuring optimal model performance.

Table 6
The impact of the number of database content rows in prompts on token overhead and average EX (%).

k-row	0	1	3	5[a]	7	9
Token (K)	491.6	592.1	676.6	759.6	839.1	912.3
Avg (EX)	82.63	84.60	85.40	85.97	86.00	**86.17**

[a] Means the selected number of parameters.

Fig. 3(b) demonstrates a strong positive correlation between the number of rows in relevant database content and the EX of SQL queries generated by the three LLMs. The performance gains become statistically insignificant when the row count exceeds five. Table 6 quantifies the trade-offs between token consumption and average EX on Spider-Dev across different numbers of rows. While including nine rows achieves the peak average EX of 86.17%, this configuration requires 20% more tokens than the five-row setup while delivering merely a 0.2% improvement in EX. Based on this cost–benefit analysis, we establish five rows as the optimal prompt configuration for balancing computational efficiency with EX.

4.5. Analysis on memorization effects and data contamination

To verify that SRSPSQL genuinely enhances the model SQL generation performance rather than relying on memorization as mentioned in [33], we analyze its predictions on the Spider development set. As shown in Table 7, based on Deepseek-v3-0324, predicted SQL and gold SQL exhibit significant structural differences while producing identical execution results. Specifically, for queries of medium or higher difficulty involving multi-table operations, SRSPSQL guides the model to generate a sub-query, whereas the gold SQL typically employs join-based formulations. These fundamental structural variations

Table 7
Examples of structural divergence between gold and predicted SQL by hardness level on Spider dev dataset.

Hardness	Query	Gold SQL	Predicted SQL
Easy	What are the country codes of countries where people use languages other than English?	SELECT DISTINCT CountryCode FROM countrylanguage WHERE LANGUAGE != 'English'	SELECT CountryCode FROM countrylanguage WHERE Language != 'English' GROUP BY CountryCode
Medium	What is the total number of unique official languages spoken in the countries that are founded before 1930?	SELECT COUNT(DISTINCT T2.Language) FROM country AS T1 JOIN countrylanguage AS T2 ON T1.Code = T2.CountryCode WHERE IndepYear < 1930 AND T2.IsOfficial = 'T'	SELECT COUNT(DISTINCT Language) FROM countrylanguage WHERE IsOfficial = 'T' AND CountryCode IN (SELECT Code FROM country WHERE IndepYear < 1930)
Hard	Show names of all high school students who do not have any friends.	SELECT name FROM Highschooler EXCEPT SELECT T2.name FROM Friend AS T1 JOIN Highschooler AS T2 ON T1.student_id = T2.id	SELECT name FROM Highschooler WHERE ID NOT IN (SELECT student_id FROM Friend)
Extra hard	Find the model of the car whose weight is below the average weight.	SELECT T1.model FROM CAR_NAMES AS T1 JOIN CARS_DATA AS T2 ON T1.MakeId = T2.Id WHERE T2.Weight < (SELECT avg(Weight) FROM CARS_DATA)	SELECT Model FROM car_names WHERE MakeId IN (SELECT Id FROM cars_data WHERE Weight < (SELECT AVG(Weight) FROM cars_data))

demonstrate that SRSPSQL effectively guides the model to comprehend natural language questions and construct corresponding query logic, rather than relying on superficial pattern memorization. Moreover, as presented in Tables 1 and 3, under identical model settings (Deepseek-v3-0324) and data splits, SRSPSQL achieves superior performance over DIN-SQL [10], DAIL-SQL [11], MAC-SQL [32] on both the Spider and Bird benchmarks. This indicates that, even in the presence of potential model memorization, SRSPSQL consistently delivers state-of-the-art results.

5. Conclusion

In this paper, we propose SRSPSQL, a dual-stage framework designed to enhance Text-to-SQL generation. SRSPSQL addresses key challenges, including natural language ambiguity and database schema redundancy, by integrating a semantic reconstruction mechanism and a dynamic schema pruning strategy. SRSPSQL not only relies on structural schema information but also incorporates more valuable database content to enrich the prompt with real-world contextual information. Considering the importance of database content information in the Text-to-SQL task, we propose a solution for the retrieval and organization of relevant database content information. Experimental results demonstrate that SRSPSQL consistently achieves state-of-the-art performance on diverse datasets, including Spider, Bird, Spider-Syn, and Spider-Realistic, outperforming both ICL and SFT baseline approaches by significant margins. SRSPSQL shows remarkable robustness in handling natural language ambiguity and database schema redundancy, maintaining high accuracy even when explicit schema references are obscured. In addition, hyper-parameter analysis reveals that exceeding five in-context examples leads to decreased execution accuracy for smaller models such as Qwen2.5-Coder-14B. This phenomenon is not unique to Text-to-SQL but aligns with the broader observation that smaller-scale LLMs are more susceptible to prompt noise and tend to experience performance degradation when overloaded with excessive contextual information. This highlights the importance of balancing prompt richness and model capacity in resource-constrained deployments. In future work, we plan to further enhance the practicality of SRSPSQL by exploring its application in more complex and dynamic real-world scenarios.

CRediT authorship contribution statement

Jianjun Lei: Writing – review & editing, Resources, Methodology, Investigation, Funding acquisition, Conceptualization. **Zhichao Li:** Writing – original draft, Validation, Investigation, Data curation, Conceptualization. **Ying Wang:** Writing – original draft, Methodology, Investigation, Data curation, Conceptualization.

Availability of data and materials

The datasets generated or analyzed during this study are available in the https://yale-lily.github.io/spider, https://github.com/ygan/Spider-Syn, https://huggingface.co/datasets/aherntech/spider-realistic and https://bird-bench.github.io/.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Key Cooperation Project of the Chongqing Municipal Education Commission (Grant No. HZ2021008) and Research Project of Graduate Education and Teaching Reform of Chongqing Municipal Education Commission (Grant No. yjg223087).

Data availability

Data will be made available on request.

References

[1] M. Han, S. Park, S. Kim, H. Kim, Bridging the gap between Text-to-SQL research and real-world applications: A unified all-in-one framework for Text-to-SQL, Knowl.-Based Syst. 306 (2024) 112697, http://dx.doi.org/10.1016/j.knosys.2024.112697.

[2] Z. Tan, X. Liu, Q. Shu, X. Li, C. Wan, D. Liu, Q. Wan, G. Liao, Enhancing Text-to-SQL capabilities of large language models through tailored promptings, in: N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, N. Xue (Eds.), Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), ELRA and ICCL, Torino, Italia, 2024, pp. 6091–6109.

[3] A. Singh, A. Shetty, A. Ehtesham, S. Kumar, T.T. Khoei, A survey of large language model-based generative AI for Text-to-SQL: Benchmarks, applications, use cases, and challenges, in: Proceedings of the 2025 IEEE 15th Annual Computing and Communication Workshop and Conference, CCWC, IEEE, Las Vegas, NV, USA, 2025, pp. 15–21, http://dx.doi.org/10.1109/CCWC62904.2025.10903689.

[4] L. Shi, Z. Tang, N. Zhang, X. Zhang, Z. Yang, A survey on employing large language models for Text-to-SQL tasks, ACM Comput. Surv. 58 (2) (2025) 1–37, http://dx.doi.org/10.1145/3737873.

[5] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Zhang, J. Fan, G. Li, N. Tang, Y. Luo, A survey of Text-to-SQL in the era of LLMs: Where are we, and where are we going? IEEE Trans. Knowl. Data Eng. 37 (10) (2025) 5735–5754, http://dx.doi.org/10.1109/TKDE.2025.3592032.

[6] C.-Y. Tai, Z. Chen, T. Zhang, X. Deng, H. Sun, Exploring chain of thought style prompting for Text-to-SQL, in: H. Bouamor, J. Pino, K. Bali (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore, 2023, pp. 5376–5393, http://dx.doi.org/10.18653/v1/2023.emnlp-main.327.

[7] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G.T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, S.O. Arik, CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in Text-to-SQL, in: The Thirteenth International Conference on Learning Representations, 2025, pp. 60385–60415.

[8] Z. Shao, S. Cai, R. Lin, Z. Ming, Enhancing Text-to-SQL with question classification and Multi-Agent collaboration, in: Findings of the Association for Computational Linguistics: NAACL 2025, Association for Computational Linguistics, Albuquerque, New Mexico, 2025, pp. 4340–4349, http://dx.doi.org/10.18653/v1/2025.findings-naacl.245.

[9] X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, L. Chen, J. Lin, D. Lou, C3: Zero-shot Text-to-SQL with ChatGPT, 2023, http://dx.doi.org/10.48550/arXiv.2307.07306, arXiv preprint arXiv:2307.07306.

[10] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed In-Context learning of Text-to-SQL with Self-Correction, in: Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23, Curran Associates Inc., Red Hook, NY, USA, 2023, pp. 1577–1586.

[11] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, Proc. VLDB Endow. 17 (5) (2024) 1132–1145, http://dx.doi.org/10.14778/3641204.3641221.

[12] Y. Dai, H. Yang, M. Hao, P. Chao, PARSQL: Enhancing Text-to-SQL through SQL parsing and reasoning, in: W. Che, J. Nabende, E. Shutova, M.T. Pilehvar (Eds.), Findings of the Association for Computational Linguistics: ACL 2025, Association for Computational Linguistics, Vienna, Austria, 2025, pp. 661–681.

[13] R. Toteja, A. Sarkar, P.M. Comar, In-Context reinforcement learning with Retrieval-Augmented generation for Text-to-SQL, in: Proceedings of the 31st International Conference on Computational Linguistics, Association for Computational Linguistics, Abu Dhabi, UAE, 2025, pp. 10390–10397, http://dx.doi.org/10.18653/v1/2025.coling-main.692.

[14] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A Large-Scale Human-Labeled dataset for complex and Cross-Domain semantic parsing and Text-to-SQL task, in: E. Riloff, D. Chiang, J. Hockenmaier, J. Tsujii (Eds.), Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 3911–3921, http://dx.doi.org/10.18653/v1/D18-1425.

[15] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K.C. Chang, F. Huang, R. Cheng, Y. Li, Can LLM already serve as a database interface? A BIg bench for Large-Scale database grounded Text-to-SQLs, in: Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23, Curran Associates Inc., Red Hook, NY, USA, 2023, pp. 1835–1862.

[16] Y. Gan, X. Chen, M. Purver, Re-appraising the schema linking for Text-to-SQL, in: A. Rogers, J. Boyd-Graber, N. Okazaki (Eds.), Findings of the Association for Computational Linguistics: ACL 2023, Association for Computational Linguistics, Toronto, Canada, 2023, pp. 835–852, http://dx.doi.org/10.18653/v1/2023.findings-acl.53.

[17] K. Zhang, X. Lin, Y. Wang, X. Zhang, F. Sun, C. Jianhe, H. Tan, X. Jiang, H. Shen, ReFSQL: A Retrieval-Augmentation framework for Text-to-SQL generation, in: H. Bouamor, J. Pino, K. Bali (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2023, Association for Computational Linguistics, Singapore, 2023, pp. 664–673, http://dx.doi.org/10.18653/v1/2023.findings-emnlp.48.

[18] J. Cen, J. Liu, Z. Li, J. Wang, SQLFixAgent: Towards semantic-accurate Text-to-SQL parsing via Consistency-Enhanced Multi-Agent collaboration, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 39, Association for the Advancement of Artificial Intelligence, New York, NY, USA, 2025, pp. 49–57, http://dx.doi.org/10.1609/aaai.v39i1.31979.

[19] M. Dong, N. Ashok Kumar, Y. Hu, A. Chauhan, C.-W. Hang, S. Chang, L. Pan, W. Lan, H. Zhu, J. Jiang, P. Ng, Z. Wang, PRACTIQ: A practical conversational Text-to-SQL dataset with ambiguous and unanswerable queries, in: L. Chiruzzo, A. Ritter, L. Wang (Eds.), Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Albuquerque, New Mexico, 2025, pp. 255–273, http://dx.doi.org/10.18653/v1/2025.naacl-long.13.

[20] Y. Luo, G. Li, J. Fan, C. Chai, N. Tang, Natural language to SQL: State of the art and open problems, Proc. VLDB Endow. 18 (12) (2025) 5466–5471, http://dx.doi.org/10.14778/3750601.3750696.

[21] H. Zhang, R. Cao, L. Chen, H. Xu, K. Yu, ACT-SQL: In-Context learning for Text-to-SQL with Automatically-Generated Chain-of-Thought, in: H. Bouamor, J. Pino, K. Bali (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2023, Association for Computational Linguistics, Singapore, 2023, pp. 3501–3532, http://dx.doi.org/10.18653/v1/2023.findings-emnlp.227.

[22] L. Nan, Y. Zhao, W. Zou, N. Ri, J. Tae, E. Zhang, A. Cohan, D. Radev, Enhancing Text-to-SQL capabilities of large language models: A study on prompt design strategies, in: H. Bouamor, J. Pino, K. Bali (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2023, Association for Computational Linguistics, Singapore, 2023, pp. 14935–14956, http://dx.doi.org/10.18653/v1/2023.findings-emnlp.996.

[23] J. Li, L. Chen, R. Cao, S. Zhu, H. Xu, Z. Chen, H. Zhang, K. Yu, Exploring schema generalizability of Text-to-SQL, in: A. Rogers, J. Boyd-Graber, N. Okazaki (Eds.), Findings of the Association for Computational Linguistics: ACL 2023, Association for Computational Linguistics, Toronto, Canada, 2023, pp. 1344–1360, http://dx.doi.org/10.18653/v1/2023.findings-acl.87.

[24] Z. Liu, S. Li, Z. Cai, X. Li, Y. Li, C. Li, X. Hu, R. Xu, M. Yang, TP-Link: Fine-grained Pre-Training for Text-to-SQL parsing with linking information, in: N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, N. Xue (Eds.), Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), ELRA and ICCL, Torino, Italia, 2024, pp. 16686–16697.

[25] H. Li, J. Zhang, C. Li, H. Chen, RESDSQL: Decoupling schema linking and skeleton parsing for Text-to-SQL, in: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence, in: AAAI'23/IAAI'23/EAAI'23, AAAI Press, Red Hook, NY, USA, 2023, pp. 1466–1474, http://dx.doi.org/10.1609/aaai.v37i11.26535.

[26] V. Zhong, C. Xiong, R. Socher, Seq2SQL: Generating structured queries from natural language using reinforcement learning, 2017, CoRR abs/1709.00103 arXiv:1709.00103.

[27] L. Wang, B. Qin, B. Hui, B. Li, M. Yang, B. Wang, B. Li, J. Sun, F. Huang, L. Si, Y. Li, Proton: Probing schema linking information from Pre-trained language models for Text-to-SQL parsing, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1889–1898, http://dx.doi.org/10.1145/3534678.3539305.

[28] A. Liu, X. Hu, L. Lin, L. Wen, Semantic enhanced Text-to-SQL parsing via iteratively learning schema linking graph, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, in: KDD '22', Association for Computing Machinery, New York, NY, USA, 2022, pp. 1021–1030, http://dx.doi.org/10.1145/3534678.3539294.

[29] D. Lee, C. Park, J. Kim, H. Park, MCS-SQL: Leveraging multiple prompts and Multiple-Choice selection for Text-to-SQL generation, in: O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. Di Eugenio, S. Schockaert (Eds.), Proceedings of the 31st International Conference on Computational Linguistics, Association for Computational Linguistics, Abu Dhabi, UAE, 2025, pp. 337–353.

[30] J. Lei, Y. Tan, Y. Wang, EMLC: An extensible multi-level correction framework for Text-to-SQL, Inf. Process. Manage. 63 (3) (2026) 104560, http://dx.doi.org/10.1016/j.ipm.2025.104560.

[31] Z. Shen, P. Vougiouklis, C. Diao, K. Vyas, Y. Ji, J.Z. Pan, Improving Retrieval-augmented Text-to-SQL with AST-based ranking and schema pruning, in: Y. Al-Onaizan, M. Bansal, Y.-N. Chen (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Miami, Florida, USA, 2024, pp. 7865–7879, http://dx.doi.org/10.18653/v1/2024.emnlp-main.449.

[32] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, Z. Li, MAC-SQL: A Multi-Agent collaborative framework for Text-to-SQL, in: O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. Di Eugenio, S. Schockaert (Eds.), Proceedings of the 31st International Conference on Computational Linguistics, Association for Computational Linguistics, Abu Dhabi, UAE, 2025, pp. 540–557.

[33] F. Ranaldi, E.S. Ruzzetti, D. Onorati, L. Ranaldi, C. Giannone, A. Favalli, R. Romagnoli, F.M. Zanzotto, Investigating the impact of data contamination of large language models in Text-to-SQL translation, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 13909–13920, http://dx.doi.org/10.18653/v1/2024.findings-acl.827.

[34] R. Luo, L. Wang, B. Lin, Z. Lin, Y. Yang, PTD-SQL: Partitioning and targeted drilling with LLMs in Text-to-SQL, in: Y. Al-Onaizan, M. Bansal, Y.-N. Chen (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Miami, Florida, USA, 2024, pp. 3767–3799, http://dx.doi.org/10.18653/v1/2024.emnlp-main.221.

[35] G. Qu, J. Li, B. Li, B. Qin, N. Huo, C. Ma, R. Cheng, Before generation, align it! A novel and effective strategy for mitigating hallucinations in Text-to-SQL generation, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 5456–5471, http://dx.doi.org/10.18653/v1/2024.findings-acl.324.

[36] G. Liu, Y. Tan, R. Zhong, Y. Xie, L. Zhao, Q. Wang, B. Hu, Z. Li, Solid-SQL: Enhanced Schema-linking based In-context learning for robust Text-to-SQL, in: O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. Di Eugenio, S. Schockaert (Eds.), Proceedings of the 31st International Conference on Computational Linguistics, Association for Computational Linguistics, Abu Dhabi, UAE, 2025, pp. 9793–9803, http://dx.doi.org/10.18653/v1/2025.coling-main.654.

[37] H. Yuan, X. Tang, K. Chen, L. Shou, G. Chen, H. Li, CogSQL: A cognitive framework for enhancing large language models in Text-to-SQL translation, in: Proceedings of the AAAI Conference on Artificial Intelligence, 39, Association for the Advancement of Artificial Intelligence, 2025, pp. 25778–25786, http://dx.doi.org/10.1609/aaai.v39i24.34770.

[38] J. Shi, B. Xu, J. Liang, Y. Xiao, J. Chen, C. Xie, P. Wang, W. Wang, Gen-SQL: Efficient Text-to-SQL by bridging natural language question and database schema with Pseudo-Schema, in: O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. Di Eugenio, S. Schockaert (Eds.), Proceedings of the 31st International Conference on Computational Linguistics, Association for Computational Linguistics, Abu Dhabi, UAE, 2025, pp. 3794–3807.

[39] J. Lee, J.-S. Lee, J. Lee, Y. Choi, J.-H. Lee, DCG-SQL: Enhancing In-Context learning for Text-to-SQL with deep contextual schema link graph, in: W. Che, J. Nabende, E. Shutova, M.T. Pilehvar (Eds.), Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 15397–15412.

[40] J. Yang, B. Hui, M. Yang, J. Yang, J. Lin, C. Zhou, Synthesizing Text-to-SQL data from weak and strong LLMs, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 7864–7875, http://dx.doi.org/10.18653/v1/2024.acl-long.425.

[41] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, H. Chen, CodeS: Towards building Open-source language models for Text-to-SQL, Proc. ACM Manag. Data 2 (3) (2024) 127, http://dx.doi.org/10.1145/3654930.

[42] M. Pourreza, D. Rafiei, DTS-SQL: Decomposed Text-to-SQL with small large language models, in: Y. Al-Onaizan, M. Bansal, Y.-N. Chen (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2024, Association for Computational Linguistics, Miami, Florida, USA, 2024, pp. 8212–8220, http://dx.doi.org/10.18653/v1/2024.findings-emnlp.481.

[43] S.K. Gorti, I. Gofman, Z. Liu, J. Wu, N. Vouitsis, G. Yu, J.C. Cresswell, R. Hosseinzadeh, MSc-SQL: Multi-Sample critiquing small language models for Text-to-SQL translation, in: L. Chiruzzo, A. Ritter, L. Wang (Eds.), Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Albuquerque, New Mexico, 2025, pp. 2145–2160, http://dx.doi.org/10.18653/v1/2025.naacl-long.107.

[44] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: J. Burstein, C. Doran, T. Solorio (Eds.), Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Association for Computational Linguistics, Minneapolis, Minnesota, 2019, pp. 4171–4186, http://dx.doi.org/10.18653/v1/N19-1423.

[45] J. Pennington, R. Socher, C. Manning, GloVe: Global vectors for word representation, in: A. Moschitti, B. Pang, W. Daelemans (Eds.), Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP, Association for Computational Linguistics, Doha, Qatar, 2014, pp. 1532–1543, http://dx.doi.org/10.3115/v1/D14-1162.

[46] C. Guo, Z. Tian, J. Tang, P. Wang, Z. Wen, K. Yang, T. Wang, A Case-Based reasoning framework for adaptive prompting in Cross-Domain Text-to-SQL, 2023, http://dx.doi.org/10.48550/arXiv.2304.13301, arXiv preprint arXiv:2304.13301.

[47] Y. Gan, X. Chen, Q. Huang, M. Purver, J.R. Woodward, J. Xie, P. Huang, Towards robustness of Text-to-SQL models against synonym substitution, in: C. Zong, F. Xia, W. Li, R. Navigli (Eds.), Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Association for Computational Linguistics, Online, 2021, pp. 2505–2515, http://dx.doi.org/10.18653/v1/2021.acl-long.195.

[48] X. Deng, A.H. Awadallah, C. Meek, O. Polozov, H. Sun, M. Richardson, Structure-Grounded pretraining for Text-to-SQL, in: K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, Y. Zhou (Eds.), Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Association for Computational Linguistics, Online, 2021, pp. 1337–1350, http://dx.doi.org/10.18653/v1/2021.naacl-main.105.